COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S2
DATE PERFORMED:	July 24, 2026	**DATE SUBMITTED:**	July 29, 2026
NAME:	Mendoza, Nathaniel B.	**INSTRUCTOR:**	Engr. Jimlord Quejado

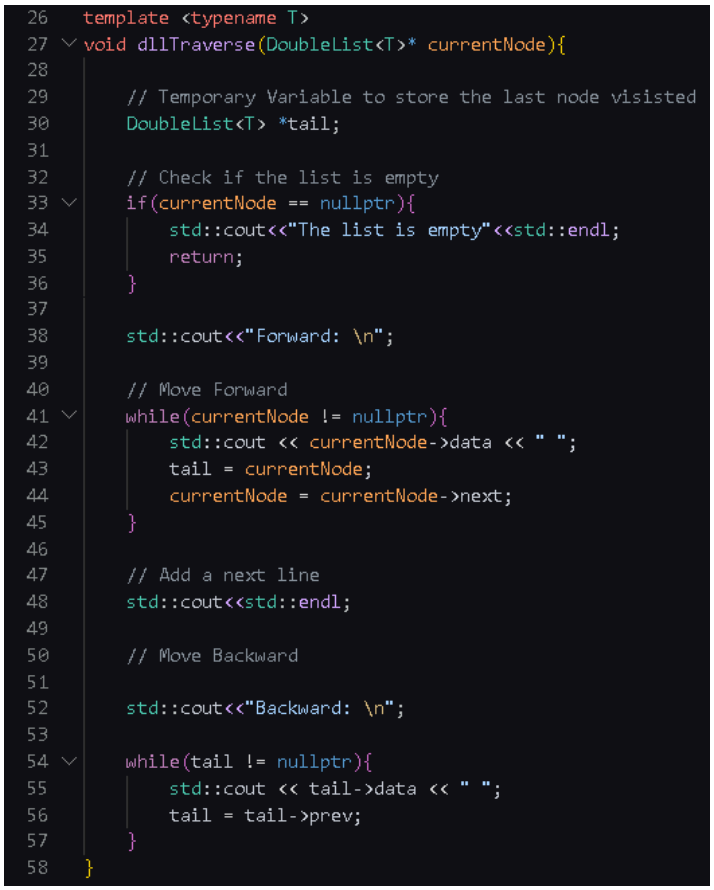
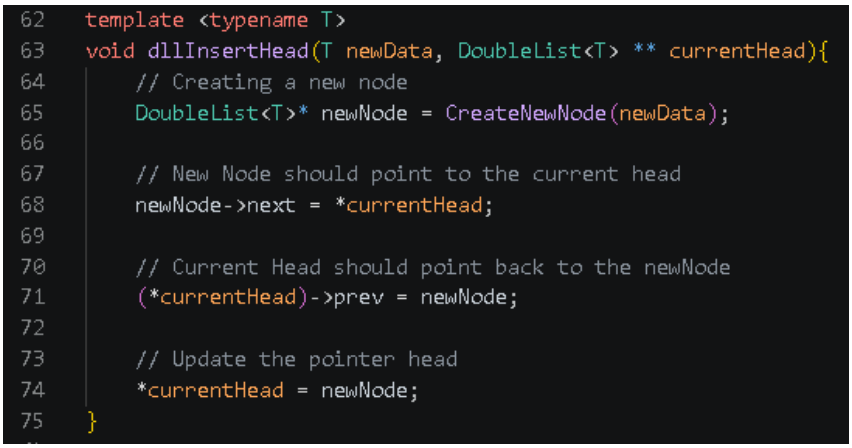
Hands-On Activity NO. 3.1 Linked List

1. PROCEDURE OUTPUT

Screenshot	implementation.cpp > ...
	1 #include <iostream>
	2 #include <utility>
	3
	4 class Node {
	5 public:
	6 char data;
	7 Node next;
	8 };
	9
	10 int main() {
	11 // Step 1
	12 Node* head = NULL;
	13 Node* second = NULL;
	14 Node* third = NULL;
	15 Node* fourth = NULL;
	16 Node* fifth = NULL;
	17 Node* last = NULL;
	18 // Step 2
	19 head = new Node;
	20 second = new Node;
	21 third = new Node;
	22 fourth = new Node;
	23 fifth = new Node;
	24 last = new Node;
	25 // Step 3
	26 head->data = 'C';
	27 head->next = second;
	28 second->data = 'P';
	29 second->next = third;
	30 third->data = 'E';
	31 third->next = fourth;
	32 fourth->data = '0';
	33 fourth->next = fifth;
	34 fifth->data = '1';
	35 fifth->next = last;
	36 // Step 4
	37 last->data = '0';
	38 last->next = nullptr;
	39 }

Discussion	The program did not produce any output because it only creates and connects the linked list nodes without including a display or traversal function.

Table 3-1. Output of Initial/Simple Implementation

Operation	Screenshot
Traversal	
Insertion at head	

```cpp
template <typename T>
void dllTraverse(DoubleList<T>* currentNode){

    // Temporary Variable to store the last node visisted
    DoubleList<T> *tail;

    // Check if the list is empty
    if(currentNode == nullptr){
        std::cout<<"The list is empty"<<std::endl;
        return;
    }

    std::cout<<"Forward: \n";

    // Move Forward
    while(currentNode != nullptr){
        std::cout << currentNode->data << " ";
        tail = currentNode;
        currentNode = currentNode->next;
    }

    // Add a next line
    std::cout<<std::endl;

    // Move Backward

    std::cout<<"Backward: \n";

    while(tail != nullptr){
        std::cout << tail->data << " ";
        tail = tail->prev;
    }
}
```

```cpp
template <typename T>
void dllInsertHead(T newData, DoubleList<T> ** currentHead){
    // Creating a new node
    DoubleList<T>* newNode = CreateNewNode(newData);

    // New Node should point to the current head
    newNode->next = *currentHead;

    // Current Head should point back to the newNode
    (*currentHead)->prev = newNode;

    // Update the pointer head
    *currentHead = newNode;
}
```

Insertion at any part of the list

```cpp
template <typename T>
void dllGeneralInsert(T newData, DoubleList<T>* prevNode){

    // Check if it is the head node (previous node is null)
    if(prevNode == nullptr){
        // If null, print "Previous node cannot be null."
        std::cout<<"Previous value cannot be null"<<std::endl;
        return;
    }
    // Create a new node
    DoubleList<T>* newNode = new DoubleList<T>;

    //4. Store data in the new node
    newNode->data = newData; // Backward
    newNode->next = prevNode->next; // Forward

    //5. Point new node to the node previous node was pointing to
    newNode->prev = prevNode; // Backward
    newNode->next = prevNode->next; // Forward

    // 6. If the node after newNode exists, update its 'prev' pointer to newNode
    prevNode->next = newNode; // Forward
    if (newNode->next != nullptr) { // Backward
        newNode->next->prev = newNode;
    }

}
```

Insertion at the end

```cpp
template <typename T>

void dllInsertEnd(T newData, DoubleList<T>* currentHead){
    // Createa new node
    DoubleList<T>* newNode = CreateNewNode(newData);

    // Traverse until we reach the last node
    while(currentHead->next != nullptr){
        currentHead = currentHead->next;
    }

    // Connect the last node to the new node
    currentHead->next = newNode;

    // Connect the new node back to the last node
    newNode->prev = currentHead;
}
```

Deletion of a node	

```cpp
// DELETE A NODE
template <typename T>
void dllDeleteNode(T findData, DoubleList<T>** head) {

    // If list is empty, nothing to delete
    if (*head == nullptr) return;

    DoubleList<T>* currentNode = *head;

    // Search for the target node
    while (currentNode != nullptr && currentNode->data != findData) {
        currentNode = currentNode->next;
    }

    // Data not found
    if (currentNode == nullptr) {
        std::cout << "The data was not found\n" << std::endl;
        return;
    }

    // If data was found then Update the NEXT pointer of the PREVIOUS node
    if (currentNode == *head) {
        // If deleting the head node, move the head pointer forward
        *head = currentNode->next;
    } else {
        // Otherwise, skip over currentNode using next pointer
        currentNode->prev->next = currentNode->next;
    }

    // Update the PREV pointer of the NEXT node
    if (currentNode->next != nullptr) {
        currentNode->next->prev = currentNode->prev;
    }

    // Free memory
    delete currentNode;
}
```

Table 3-2.Code for the List Operations

a	**Source Code**	

```cpp
/* List Traversal */
template <typename N>
void ListTraversal(SingleList<N>* head) {

    while (head != nullptr) {
        // PRINT data of N
        std::cout << head->data << "-->";

        // GO NEXT NODE N := next
        head = head->next;
    }

    std::cout << std::endl;
}
```

	Console	

```
Testing of traversal
J-->P-->C-->L-->
```

b	**Source Code**	

```cpp
/* Insert at Head */
template <typename N>
void sllinsertHead(N newData, SingleList<N>** currentHead) {

    // 1. Allocate memory
    SingleList<N>* newNode = new SingleList<N>;

    // 2. Put data into the new node
    newNode->data = newData;

    // 3. Set next to the current head
    newNode->next = *currentHead;

    // 4. Make the new node the new head
    *currentHead = newNode;
}
```

	Console	

```
Testing of sllinsertHead
Q-->J-->P-->C-->L-->
```

c

Source Code

```
46  /* General Insert */
47  template <typename N>
48  void sllGeneralInsert(N newData, SingleList<N>* prevNode) {
49
50      if (prevNode == nullptr) {
51          std::cout << "Previous Value cannot be null" << std::endl;
52          return;
53      }
54
55      SingleList<N>* newNode = new SingleList<N>;
56
57      newNode->data = newData;
58
59      newNode->next = prevNode->next;
60
61      prevNode->next = newNode;
62  }
```

Console

```
Testing of sllGeneralInsert
Q-->J-->P-->X-->C-->L-->
```

d

Source Code

```
65  template <typename N>
66  void sllinsertEnd(N newData, SingleList<N>** head) {
67
68      // 1. Allocate a new node
69      SingleList<N>* newNode = new SingleList<N>;
70
71      // 2. Store data in new node
72      newNode->data = newData;
73
74      // 3. Point to null
75      newNode->next = nullptr;
76
77      // 4. Dereference the head node
78      SingleList<N>* currentNode = *head;
79
80      // If the list is empty
81      if (currentNode == nullptr) {
82          *head = newNode;
83          return;
84      }
85
86      // 5. Traverse the list
87      while (currentNode->next != nullptr) {
88          currentNode = currentNode->next;
89      }
90
91      // 6. Point the next of the current node to the new node
92      currentNode->next = newNode;
93  }
```

Console

```
Testing of sllinsertEnd
Q-->J-->P-->X-->C-->L-->U-->
```

e

Source Code

```
/* Delete Node */
template <typename N>
    void sllDeleteNode(N findData, SingleList<N>** head) {

    // If there is nothing to delete
    if (*head == nullptr) {
        return;
    }

    SingleList<N>* currNode = *head;
    SingleList<N>* prevNode = nullptr;

    // Search for the node
    while (currNode != nullptr && currNode->data != findData) {

        prevNode = currNode;

        currNode = currNode->next;
    }

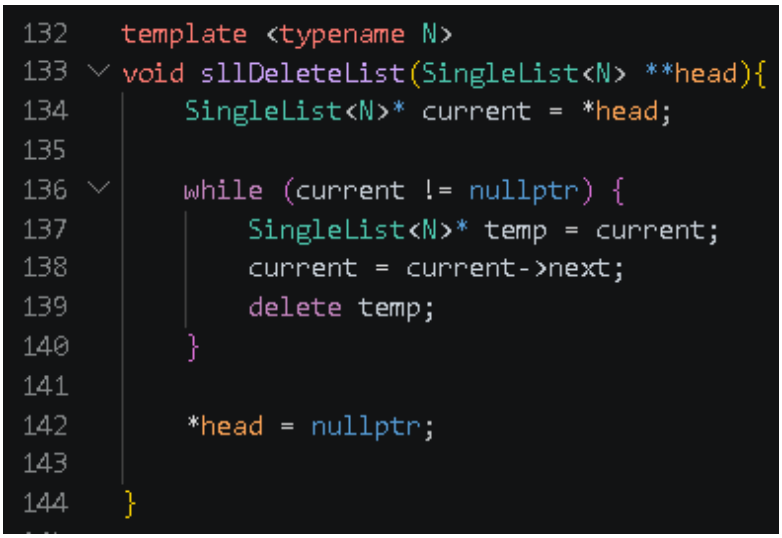
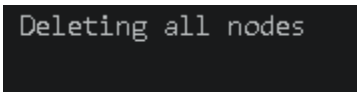
    // If data was not found
    if (currNode == nullptr) {
        std::cout << "The data is not found\n";
        return;
    }

    // If the head node is being deleted
    if (prevNode == nullptr) {
        *head = currNode->next;
    }
    else {
        prevNode->next = currNode->next;
    }

    delete currNode;
}
```

Console

```
Testing of deleting the node
Q-->J-->P-->X-->C-->L-->
```

f	**Source Code**	
	Console	

```
132  template <typename N>
133  void sllDeleteList(SingleList<N> **head){
134      SingleList<N>* current = *head;
135
136      while (current != nullptr) {
137          SingleList<N>* temp = current;
138          current = current->next;
139          delete temp;
140      }
141
142      *head = nullptr;
143
144  }
```

```
Deleting all nodes
```

Table 3-3. Code and Analysis for Singly Linked Lists

Screenshots	Analysis


``
// DELETE A NODE
template <typename T>
void dllDeleteNode(T findData, DoubleList<T>** head) {
 	
    // If list is empty, nothing to delete
    if (*head == nullptr) return;
 	
    DoubleList<T>* currentNode = *head;
 	
    // Search for the target node
    while (currentNode != nullptr && currentNode->data != findData) {
        currentNode = currentNode->next;
    }
 	
    // Data not found
    if (currentNode == nullptr) {
        std::cout << "The data was not found\n" << std::endl;
        return;
    }
 	
    // If data was found then Update the NEXT pointer of the PREVIOUS node
    if (currentNode == *head) {
        // If deleting the head node, move the head pointer forward
        *head = currentNode->next;
    } else {
        // Otherwise, skip over currentNode using next pointer
        currentNode->prev->next = currentNode->next;
    }
 	
    // Update the PREV pointer of the NEXT node
    if (currentNode->next != nullptr) {
        currentNode->next->prev = currentNode->prev;
    }
 	
    // Free memory
    delete currentNode;
}
``
	This function lets you slot a new piece of data right behind an existing one in your list. It creates a new node, wires its front and back pointers to the adjacent nodes, and then updates those neighbors to point back to it.

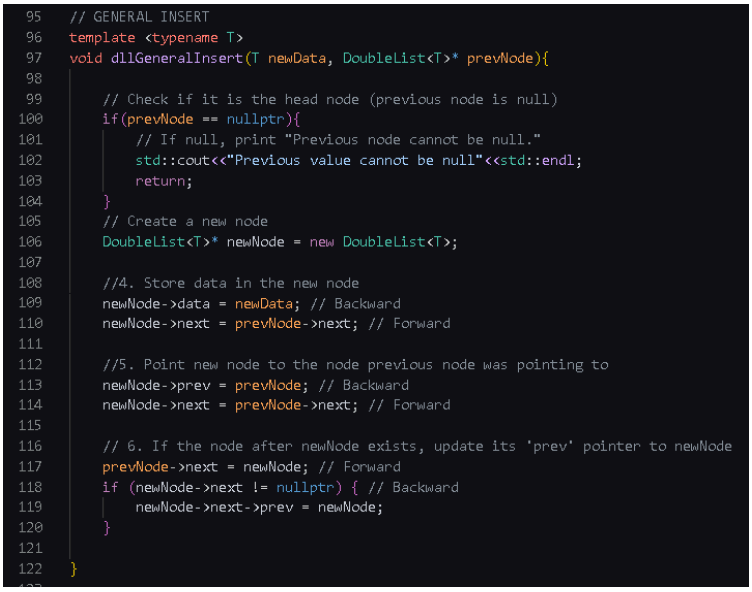
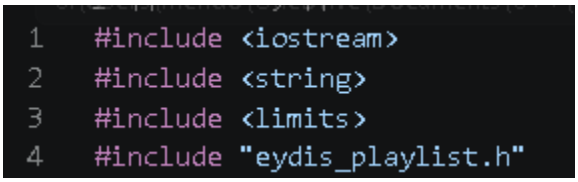
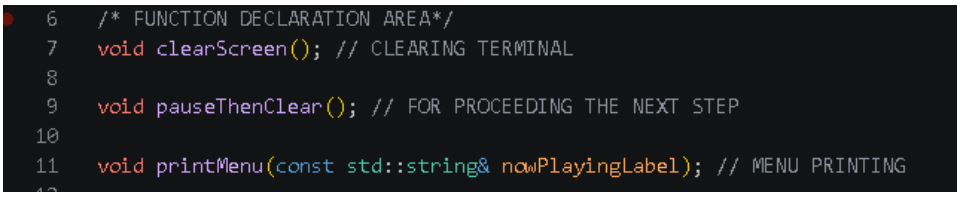

``
// GENERAL INSERT
template <typename T>
void dllGeneralInsert(T newData, DoubleList<T>* prevNode){
 	
    // Check if it is the head node (previous node is null)
    if(prevNode == nullptr){
        // If null, print "Previous node cannot be null."
        std::cout<<"Previous value cannot be null"<<std::endl;
        return;
    }
    // Create a new node
    DoubleList<T>* newNode = new DoubleList<T>;
 	
    //4. Store data in the new node
    newNode->data = newData; // Backward
    newNode->next = prevNode->next; // Forward
 	
    //5. Point new node to the node previous node was pointing to
    newNode->prev = prevNode; // Backward
    newNode->next = prevNode->next; // Forward
 	
    // 6. If the node after newNode exists, update its 'prev' pointer to newNode
    prevNode->next = newNode; // Forward
    if (newNode->next != nullptr) { // Backward
        newNode->next->prev = newNode;
    }
 	
}
``
	This function hunts down a specific item in your list, removes it, and glues the broken ends back together. It scans through the list from the start, bypasses the target node by linking its neighbors together, and then wipes the removed node from memory.

Table 3-4. Modified Operations for Doubly Linked Lists

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

main.cpp	
Header File	
Analysis	The header file includes the libraries and playlist header needed by the program. It makes sure the program knows the functions, classes, and features before running.
Global Declaration	
Analysis	The global declarations list the functions that will be used later in the program. This keeps the code organized and lets the compiler know what each function does.
Main Program	

```
#include <iostream>
#include <string>
#include <limits>
#include "eydis_playlist.h"
```

```
/* FUNCTION DECLARATION AREA*/
void clearScreen(); // CLEARING TERMINAL

void pauseThenClear(); // FOR PROCEEDING THE NEXT STEP

void printMenu(const std::string& nowPlayingLabel); // MENU PRINTING
```

```
int main() {
    SongNode<std::string>* playlist = nullptr;
    SongNode<std::string>* nowPlaying = nullptr;

    int choice = 0;

    do {
        std::string nowPlayingLabel = (nowPlaying == nullptr) ? "(none)" : nowPlaying->data;

        printMenu(nowPlayingLabel);
        std::cin >> choice;

        // Discard the leftover newline so getline works cleanly next
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

        switch (choice) {
            case 1: {
                std::cout << "Enter the song name to add: ";
                std::string songName;
                std::getline(std::cin, songName);

                playlistInsert(&playlist, songName);

                // If this was the first song added, it becomes "now playing"
                if (nowPlaying == nullptr) {
                    nowPlaying = playlist;
                }

                std::cout << "\nAdded \"" << songName << "\" to the playlist." << std::endl;
                playAll(playlist);
                break;
            }
```

```
        case 2: {
            std::cout << "Enter the song name to remove: ";
            std::string songName;
            std::getline(std::cin, songName);

            // If we're about to delete the song that's "now playing",
            // move now-playing off of it first
            if (nowPlaying != nullptr && nowPlaying->data == songName) {
                nowPlaying = (nowPlaying->next != nowPlaying) ? nowPlaying->next : nullptr;
            }

            playlistRemove(&playlist, songName);

            if (playlist == nullptr) {
                nowPlaying = nullptr;
            }

            std::cout << "\nUpdated playlist:" << std::endl;
            playAll(playlist);
            break;
        }

        case 3: {
            std::cout << "\nFull playlist:" << std::endl;
            playAll(playlist);
            break;
        }
```

```
        case 4: {
            if (nowPlaying == nullptr) {
                std::cout << "\nPlaylist is empty - nothing to play." << std::endl;
            } else {
                nextSong(&nowPlaying);
                std::cout << "\nSkipped forward. Now playing: " << nowPlaying->data << std::endl;
            }
            break;
        }

        case 5: {
            if (nowPlaying == nullptr) {
                std::cout << "\nPlaylist is empty - nothing to play." << std::endl;
            } else {
                previousSong(&nowPlaying);
                std::cout << "\nWent back. Now playing: " << nowPlaying->data << std::endl;
            }
            break;
        }

        case 6: {
            std::cout << "\nExiting playlist. Goodbye!" << std::endl;
            break;
        }

        default: {
            std::cout << "\nInvalid choice, please pick 1-6." << std::endl;
            break;
        }
    }
```

Analysis	The main() function controls the whole playlist program by showing the menu and waiting for the user's choice. It keeps repeating until the user exits, making the playlist interactive.

Function Definition	
Analysis	This part is where it approaches the global declaration where these are defined thoroughly and refers to it when called.

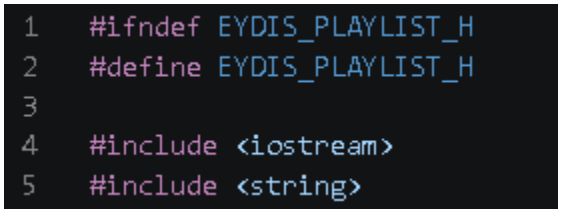
```
/* FUNCTION DECLARATION AREA */

void clearScreen() { // CLEARNING TERMINAL FOR GOOD OUTPUT I GUESS?
#ifdef _WIN32
    system("cls");
#else
    system("clear");
#endif
}

void pauseThenClear() {
    std::cout << "\nPress ENTER to continue...";
    std::cin.get();
    clearScreen();
}

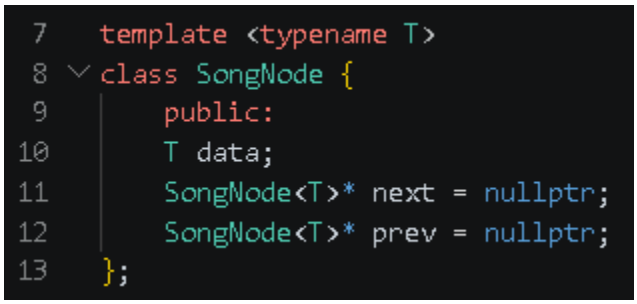
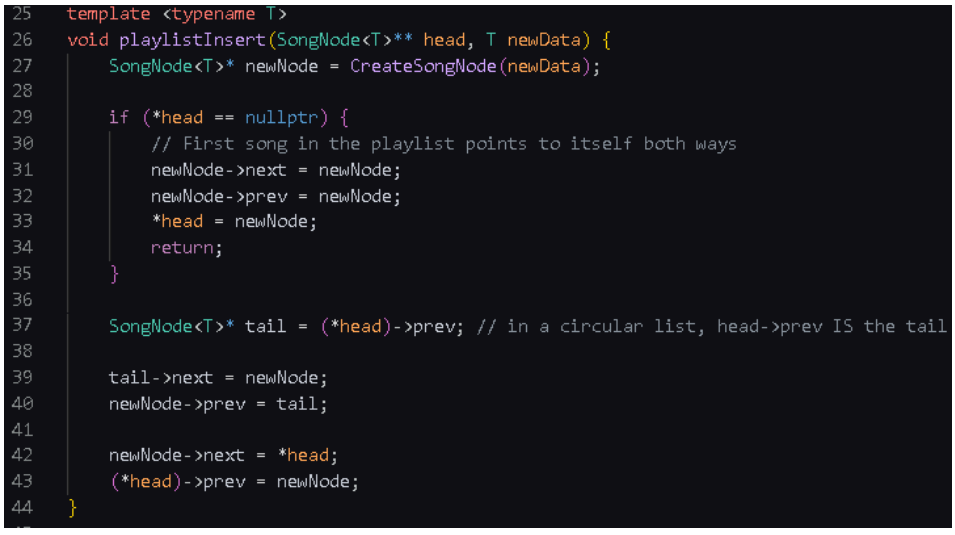
void printMenu(const std::string& nowPlayingLabel) {
    std::cout << "===== SONG PLAYLIST =====" << std::endl;
    std::cout << "Now playing: " << nowPlayingLabel << std::endl;
    std::cout << "-------------------------" << std::endl;
    std::cout << "1. Add a song" << std::endl;
    std::cout << "2. Remove a song" << std::endl;
    std::cout << "3. Play all (show full playlist)" << std::endl;
    std::cout << "4. Next song" << std::endl;
    std::cout << "5. Previous song" << std::endl;
    std::cout << "6. Exit" << std::endl;
    std::cout << "Choice: ";
}
```

Table 4. Playlist

eydis_playlist.h	
Header File	
Analysis	The header file includes the libraries and playlist header needed by the program. It makes sure the program knows the functions, classes, and features before running.

```
#ifndef EYDIS_PLAYLIST_H
#define EYDIS_PLAYLIST_H

#include <iostream>
#include <string>
```

Class	
Analysis	The class groups the song data and related operations into one structure. It makes the program easier to manage because everything about the playlist is kept together.
SongNode	
Analysis	SongNode stores one song and a pointer to the next song in the playlist. You can imagine it as one box connected to another box, forming a chain of songs.
Insert Playlist	
Analysis	This function adds a new song to the playlist and links it to the correct position. The new node becomes part of the chain without affecting the other songs.

Remove Song	

```cpp
template <typename T>
void playlistRemove(SongNode<T>** head, T findData) {
    if (*head == nullptr) {
        std::cout << "The playlist is empty" << std::endl;
        return;
    }

    SongNode<T>* current = *head;
    bool found = false;

    do {
        if (current->data == findData) {
            found = true;
            break;
        }
        current = current->next;
    } while (current != *head);

    if (!found) {
        std::cout << "Song not found in playlist" << std::endl;
        return;
    }

    if (current->next == current) {
        // Only one song in the playlist
        *head = nullptr;
    } else {
        current->prev->next = current->next;
        current->next->prev = current->prev;

        if (current == *head) {
            *head = current->next;
        }
    }
```

```cpp
    delete current;
}
```

Analysis	This function searches for the selected song and removes it from the linked list. It reconnects the remaining songs so the playlist still works after the song is deleted.

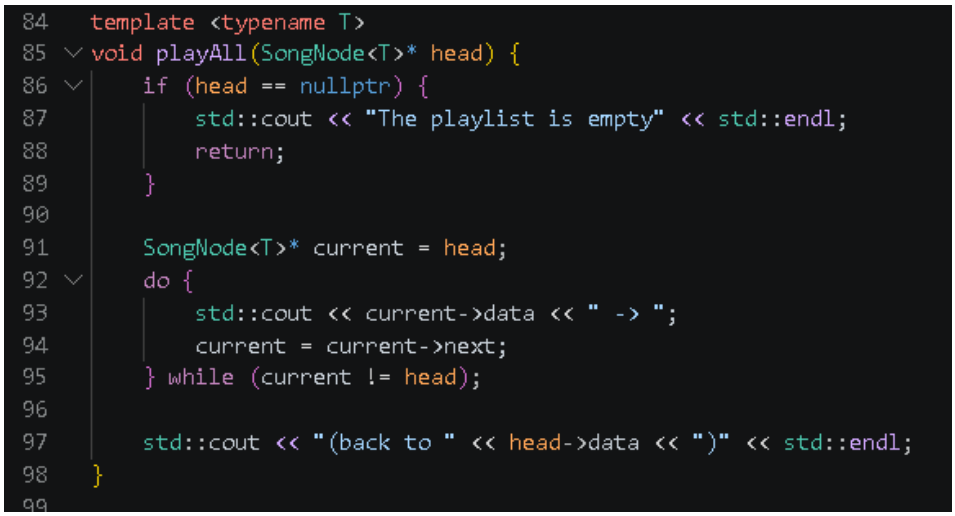
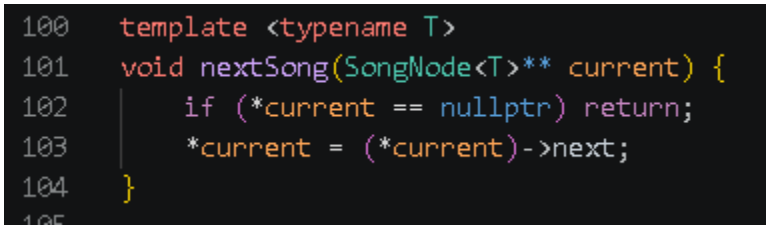
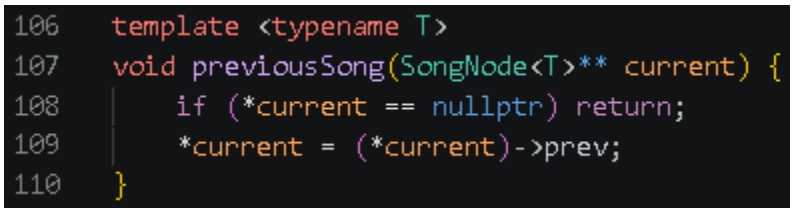
Play All / Display	
Analysis	This function goes through the linked list one by one and displays every song. It starts from the first song and stops when there are no more connected nodes.
Next Song	
Analysis	This function moves the current song pointer to the next node in the playlist. It makes the next linked song become the one that is currently playing.
Previous Song	
Analysis	This function returns to the song before the current one if it exists. It changes the current pointer back so the previous song becomes the active song.

Table 5. Main Program

CONCLUSION
Through this activity, I learned that linked lists work like a chain where each node is connected to the next one, making it easier to insert or remove data without changing the whole structure. which helped me understand why every link between nodes is important.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.